

Desafio em Grupo — Programação Orientada a Objetos

Curso de Java Fundamentals • SENAC

Grupos de 3 a 5 pessoas • Duração: 3 horas • Entrega no final da aula

Contexto do Negócio

Vocês foram contratados como squad de desenvolvimento de uma rede de bibliotecas comunitárias chamada **LeitorLivre**, que opera em cinco bairros da cidade. A rede compartilha o acervo entre unidades, atende leitores de todas as idades e enfrenta um problema recorrente: o controle de empréstimos e devoluções hoje é feito em planilhas, e cada bibliotecário aplica regras diferentes sobre quem pode pegar o quê e por quanto tempo.

A coordenação da rede precisa de um **sistema unificado de empréstimos** em Java que elimine essa inconsistência. O squad de vocês será responsável por **modelar o domínio** e implementar a lógica central — sem banco de dados, sem interface gráfica, sem framework. Apenas classes Java rodando em um **main** que demonstre os cenários funcionando.

Problemática

A coordenação reuniu as **regras de negócio** levantadas com os bibliotecários. O desafio do squad é **traduzir essas regras para uma modelagem orientada a objetos coerente**, decidindo em conjunto como representar entidades, comportamentos, hierarquias e validações.

#	Regra de Negócio
1	O acervo possui três tipos de itens: Livro , Revista e HQ . Todos compartilham título, código único e ano de publicação, mas cada tipo tem características e regras próprias.
2	Cada item pertence a uma unidade (bairro). Itens podem ser emprestados em qualquer unidade, mas devem ser devolvidos na unidade de origem.
3	Existem dois perfis de leitor: Comum (pode pegar até 3 itens simultaneamente) e Premium (paga mensalidade e pode pegar até 7 itens).
4	O prazo de empréstimo varia por tipo de item: Livro = 14 dias; Revista = 7 dias; HQ = 10 dias.
5	Leitor Premium tem 50% a mais de prazo em qualquer tipo de item (arredondado para cima).
6	Devolução em atraso gera multa de R\$ 1,50 por dia por item. Leitor com multa pendente não pode realizar novos empréstimos até quitar.
7	Revistas e HQs não podem ser renovadas. Livros podem ser renovados uma única vez , se não houver reserva de outro leitor para o mesmo exemplar.
8	Cada item possui uma classificação de faixa etária : Livre (qualquer idade), Infantil (até 11 anos), Juvenil (12 a 17 anos) ou Adulto (18 anos ou mais). O sistema deve impedir o empréstimo de um item para um leitor fora da faixa permitida — itens Infantil ou Juvenil só podem ser emprestados para leitores dentro da faixa indicada; itens Adulto exigem leitor maior de idade.
9	O sistema deve emitir um extrato de empréstimos ativos de um leitor, mostrando cada item, data limite de devolução e situação (em dia / atrasado / pode renovar).

Objetivo da Atividade

Ao final das 3 horas, o grupo deve entregar um projeto Java que **traduza fielmente as regras acima em código orientado a objetos**, com uma classe **Main** demonstrando os cenários obrigatórios listados adiante. Não existe uma única resposta certa — será avaliado o quanto a modelagem reflete as regras e o vocabulário do domínio.

Cenários Obrigatórios para Demonstrar no Main

O grupo deve provar que o sistema funciona executando, no mínimo, os seguintes cenários. Cada cenário deve imprimir no console o que aconteceu e por quê.

Cenário A — Cadastro e empréstimo simples

Cadastrar 2 unidades, 4 itens (um de cada tipo, incluindo pelo menos um com classificação Infantil ou Juvenil) e 2 leitores (1 Comum e 1 Premium). Realizar empréstimos válidos para ambos.

Cenário B — Limite de empréstimos atingido

Tentar emprestar um quarto item para o leitor Comum. O sistema deve recusar com mensagem clara.

Cenário C — Prazo diferenciado do Premium

Emprestar o mesmo tipo de item para um leitor Comum e um Premium e mostrar que o prazo de devolução do Premium é maior.

Cenário D — Restrição por faixa etária

Tentar emprestar um item classificado como Adulto para um leitor de 14 anos, e também tentar emprestar um item Infantil para um leitor de 30 anos. Ambas as operações devem ser recusadas com mensagem clara.

Cenário E — Devolução com atraso

Simular uma devolução após o prazo e mostrar o cálculo da multa. Em seguida, tentar um novo empréstimo com a multa pendente e mostrar que é recusado.

Cenário F — Renovação

Renovar um livro com sucesso, tentar renová-lo uma segunda vez (deve falhar) e tentar renovar uma revista (deve falhar).

Cenário G — Extrato de empréstimos

Imprimir o extrato de um leitor com pelo menos 3 empréstimos ativos em estados diferentes (em dia, atrasado, pode renovar).

Divisão de Papéis no Squad

Para grupos de **3 a 5 pessoas**, sugerimos a seguinte divisão de responsabilidades. Em grupos de 3, acumular papéis afins. O código deve ser integrado pelo grupo todo — papéis organizam o trabalho, não isolam pessoas.

Papel	Responsabilidade Principal
Arquiteto(a) do Domínio	Lidera o desenho de classes, hierarquia e relacionamentos. Garante que a modelagem reflete as regras.
Dev de Acervo & Unidade	Implementa Item, Livro, Revista, HQ e Unidade. Cuida das regras específicas de cada tipo de item.
Dev de Leitores & Perfis	Implementa Leitor (Comum e Premium), limites, multas e regras de elegibilidade.
Dev de Empréstimo	Implementa a lógica central de empréstimo, devolução, renovação, prazos e extrato.
QA & Integrador(a)	Escreve os cenários no Main, valida saídas, identifica regras não atendidas e organiza a apresentação final.

Sugestão de Distribuição das 3 Horas

Tempo	Etapa	Objetivo da Etapa
0:00 - 0:30	Descoberta & Modelagem	Ler as regras juntos, identificar entidades e desenhar o diagrama de classes no papel ou quadro.
0:30 - 1:45	Implementação Paralela	Cada dupla/trio implementa sua parte do modelo, alinhando assinaturas via combinação prévia.
1:45 - 2:15	Integração	Unir o código, resolver conflitos de assinatura e fazer o primeiro compile end-to-end.
2:15 - 2:45	Cenários no Main & Ajustes	Escrever os 7 cenários obrigatórios, validar saídas e corrigir regras que falharem.

Tempo	Etapa	Objetivo da Etapa
2:45 - 3:00	Apresentação	Cada grupo apresenta em ~5 min: decisões de modelagem, dificuldades e demonstração rodando.

Critérios de Avaliação

A nota do grupo será composta pelos critérios abaixo. Todos os integrantes recebem a mesma nota base, com possibilidade de ajuste individual se houver participação desigual evidente.

Critério	O que será observado	Peso
Modelagem do domínio	Entidades, hierarquias e relacionamentos refletem fielmente as regras descritas.	25%
Encapsulamento e coesão	Atributos protegidos, métodos com responsabilidade única, regras não vazam para o chamador.	15%
Herança e polimorfismo	Uso justificado para representar tipos de item e perfis de leitor; comportamentos específicos no lugar certo.	15%
Validação de regras	Operações inválidas são recusadas com mensagem clara (limite, multa, faixa etária, renovação).	20%
Cenários funcionando	Todos os 7 cenários do Main rodam, imprimem saída coerente e cobrem o comportamento esperado.	15%
Trabalho em equipe	Divisão de tarefas, integração do código e clareza na apresentação final do grupo.	10%

Entregáveis

■ Código-fonte Java compilável

Projeto organizado em pacotes coerentes (sugestão: domínio, leitor, item, emprestimo). Sem dependência externa.

■ Classe Main executável

Demonstra todos os 7 cenários obrigatórios em sequência, com prints explicativos no console.

■ Diagrama de classes (rascunho)

Foto do quadro ou imagem simples. Não precisa ser UML formal — o objetivo é mostrar como o grupo pensou o domínio.

■ Apresentação oral de 5 minutos

Cada integrante fala ao menos uma decisão técnica que tomou e qual regra ela atende.

Regras de Ouro do Squad

✓	Comecem pela modelagem no papel . Não abram a IDE nos primeiros 20 minutos.
✓	Combinem assinaturas de métodos antes de dividir a implementação.
✓	Use o vocabulário do negócio nos nomes de classes e métodos (ex: <i>realizarEmprestimo</i> , não <i>doStuff</i>).
✗	Não usem banco de dados, interface gráfica, frameworks ou bibliotecas externas.
✗	Não copiem soluções prontas da internet. O aprendizado está no debate interno do grupo.
✗	Não deixem ninguém do grupo sem entender a solução. Todos devem conseguir explicar.

Importante: esta atividade não tem uma resposta única. Decisões diferentes de modelagem podem ser igualmente válidas, desde que **respeitem todas as regras de negócio** e sejam **defendidas pelo grupo** durante a apresentação. O professor poderá questionar qualquer integrante sobre qualquer parte do código.